

COMUNICAÇÃO EM SISTEMAS AUTÔNOMOS CRÍTICOS

BIBLIOTECA DE COMUNICAÇÃO CONFIÁVEL PARA SISTEMAS AUTÔNOMOS CRÍTICOS, CENÁRIO DE VEÍCULOS AUTÔNOMOS

< P1 >

GRUPO M10:

ANDRÉ FILIPE MARTINS

ARTUR TRIBECK FERREIRA TOMAZ

JOÃO PEDRO DE OLIVEIRA ANDERS



VISÃO GERAL DA ARQUITETURA

CADA CAMADA DELEGA PARA A DE BAIXO; NENHUMA PULA ETAPA NEM CONHECE DETALHES DA CAMADA ANTERIOR.



*Essa mesma pilha atende ao requisito de API unificada: veículos e componentes usam exatamente o mesmo Communicator, sem distinção entre comunicação interna ou entre VMs.



ENGINE

- **Recepção é orientada a sinal POSIX (SIGIO) — não usa thread dedicada**
- **Ao chegar o sinal, o handler chama `drain(): recvfrom()` em loop até esvaziar o buffer do kernel, protegendo contra rajadas de pacotes**
- **Envio é síncrono: `send()` → uma única syscall, `sendto()`**
- **Erros de rede (ex: interface caiu) ficam em contadores — nunca derrubam o processo**

RAW_SOCKET_ENGINE

```
- _sockfd : int
- _ifindex : int
- _address : Ethernet::Address
- _protocol : uint16_t
- _armed : sig_atomic_t
- _rx_error, _rx_errors : sig_atomic_t

+ engine_send(frame, size) : int
+ engine_start() : bool
+ engine_stop() : void
+ engine_valid() : bool
- signal_handler(signo) : void
  {static — instalado via sigaction(SIGIO)}
- drain() : void
```



NIC

- Pool fixo de buffers: `alloc()` já monta o cabeçalho (destino, nosso MAC, EtherType)
- `send(Buffer*)` → `Engine::engine_send()`; libera o buffer sempre, sucesso ou falha
- `handle()` é a porta de entrada — roda dentro do signal handler, nada de `printf/new` ali

NIC<ENGINE>

```
- _statistics : Statistics
- _buffer : Buffer[BUFFER_SIZE]

+ NIC()
+ ~NIC()
+ send(dst, prot, data, size) : int
+ alloc(dst, prot, size) : Buffer*
+ send(buf) : int
+ free(buf) : void
+ unmarshal(buf, src, dst, data, size) : int
+ address() : const Address&
+ valid(): bool
- handle(frame, size) : void
  {override — chamado pela Engine}
```



PROTOCOL

- **Multiplexa UMA NIC entre vários interlocutores — a NIC separa por EtherType, o Protocol separa por Porta**
- **Duas caras: observador da NIC (roda no signal handler, não bloqueia) e observado pelos Communicators (via Concurrent_Observed, com semáforo)**

PROTOCOL<NIC_T>

```
- _nic : NIC_T*  
- _observed :  
  Concurrent_Observed<Buffer, Port>
```

```
+ Protocol(nic)  
+ ~Protocol()  
+ send(from, to, data, size) : int  
+ receive(buf, from, data, size) : int  
+ attach(obs, address) : void  
+ detach(obs, address) : void  
- update(prot, buf) : void  
  {override — chamado pela NIC, no signal handler}
```



COMMUNICATOR

- Único ponto de contato da aplicação
- **receive()** bloqueia no semáforo do **Concurrent_Observer**, não em **recvfrom()**, o signal handler nunca espera pela aplicação
- **send()** sempre em broadcast, sem endereçamento explícito; retorno **true** só garante entrega ao kernel, não que alguém recebeu

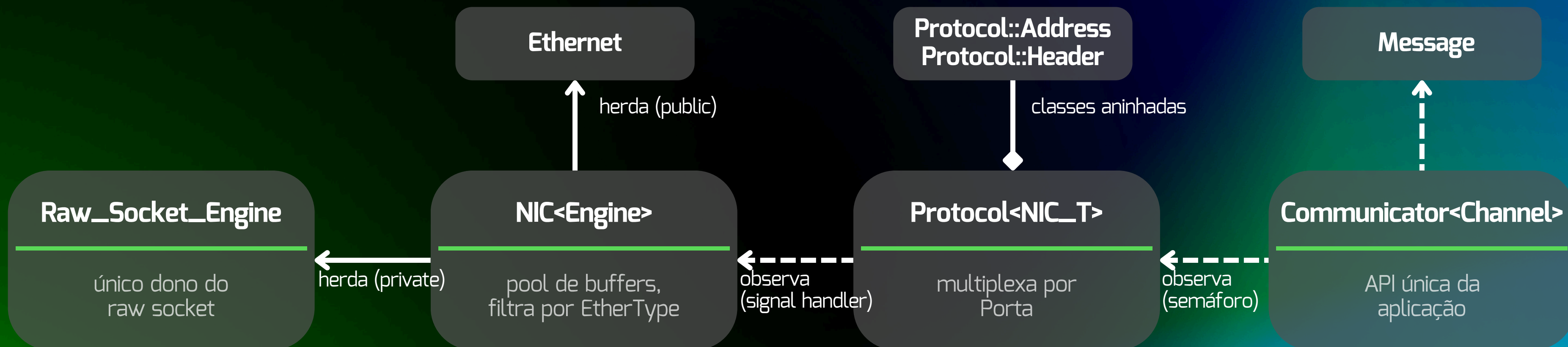
COMMUNICATOR<CHANNEL>

```
- _channel : Channel*  
- _address : Address  
  
+ Communicator(channel, address)  
+ ~Communicator()  
+ send(message) : bool  
+ receive(message) : bool  
+ address() : const Address&  
- update(cond, buf) : void  
  {override — chamado pelo Protocol, no signal handler}
```



DIAGRAMA DE CLASSES

VISÃO CONSOLIDADA



—————> herança

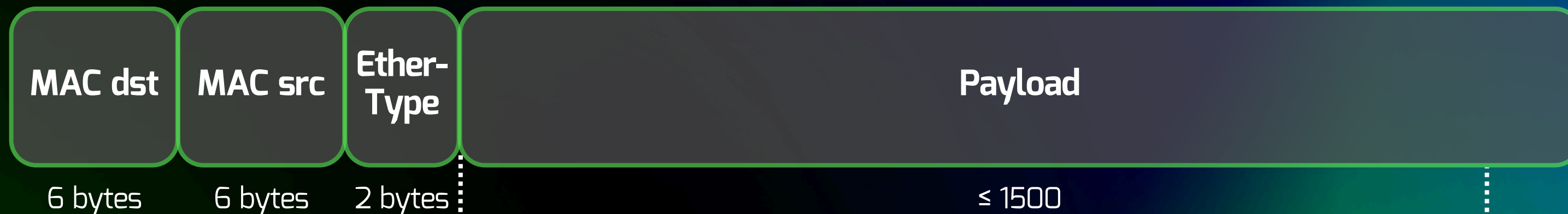
- - - - -> dependência (usa/observa)

—————> Composição



FORMATO DO FRAME

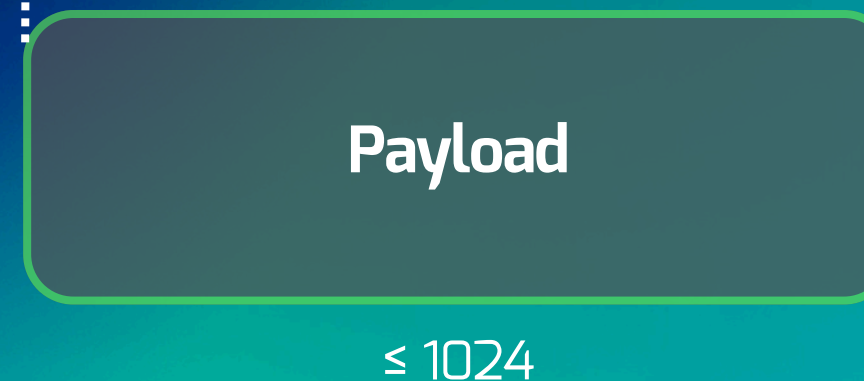
Ethernet Frame



Protocol Packet



Message

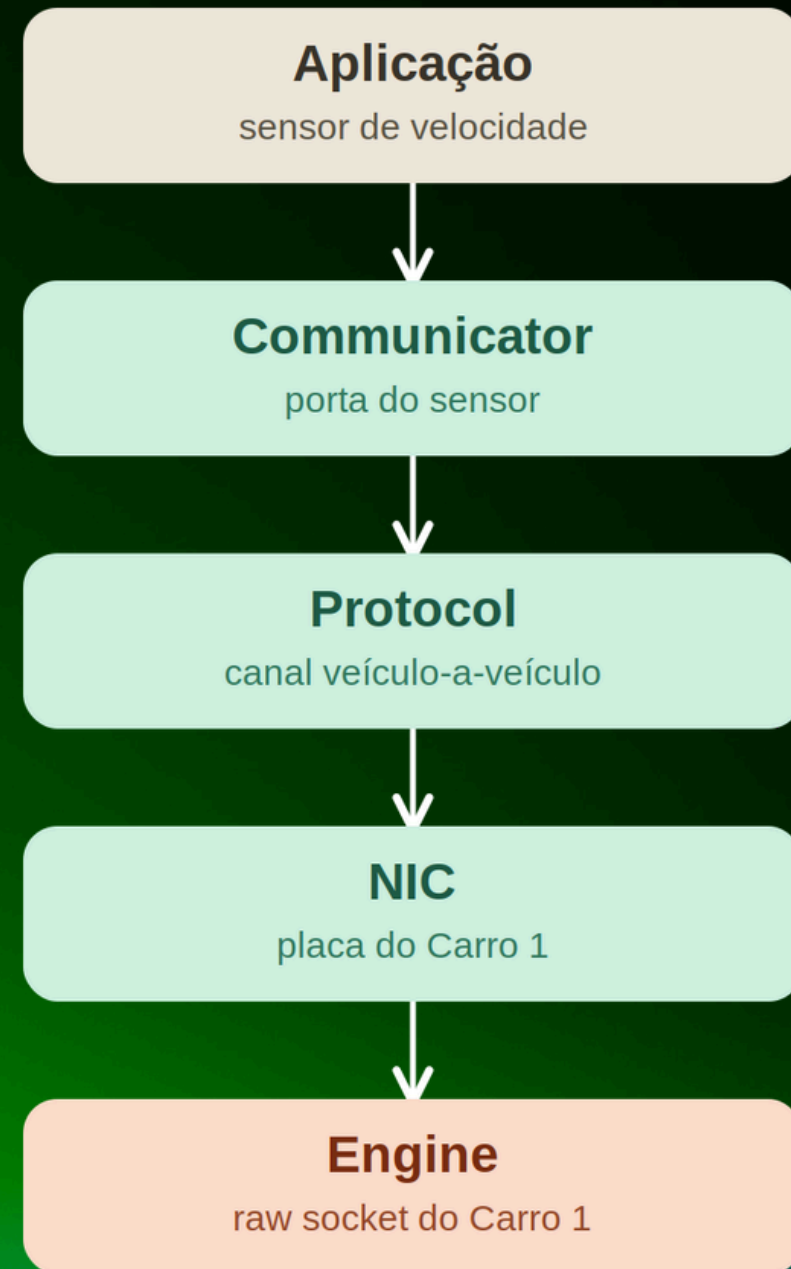


FLUXO DE COMUNICAÇÃO

VISÃO GERAL

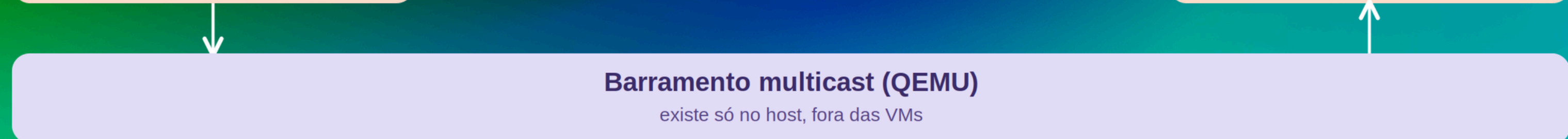
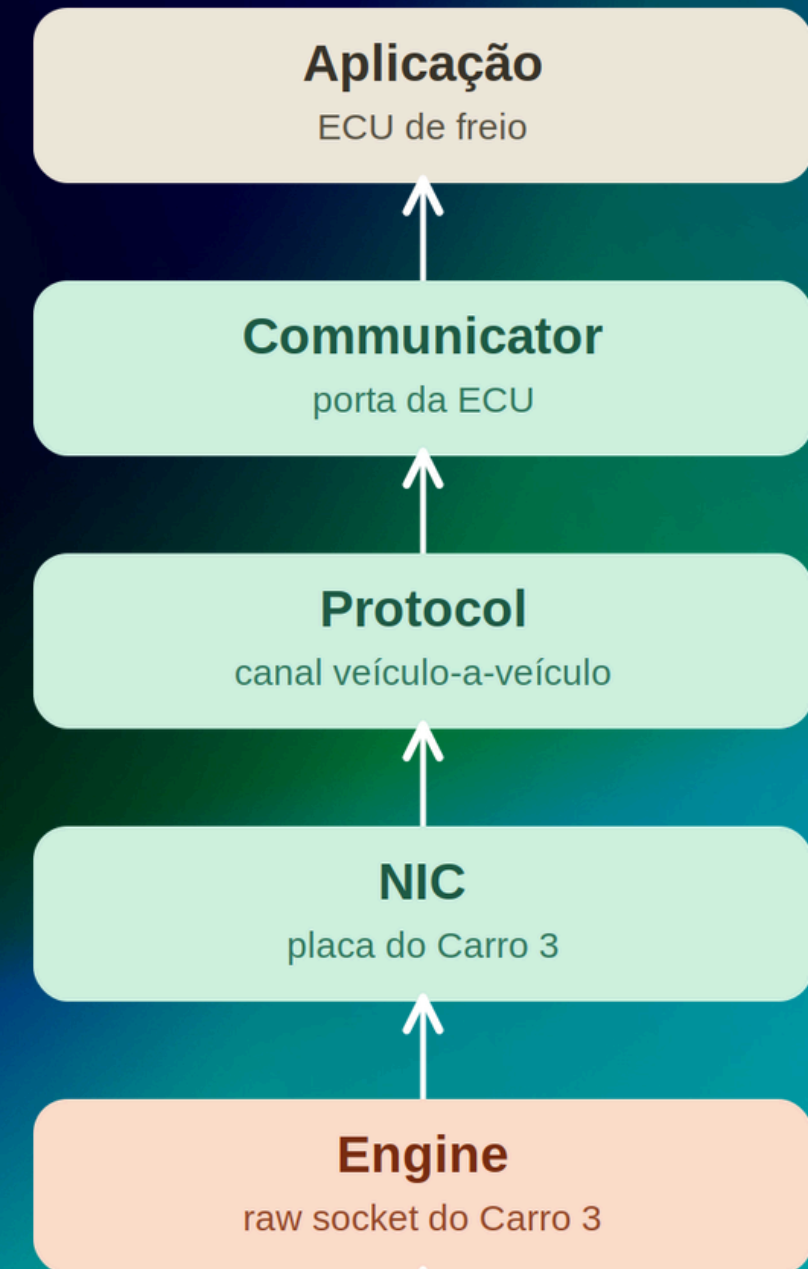
Carro 1 — remetente

envio ▼



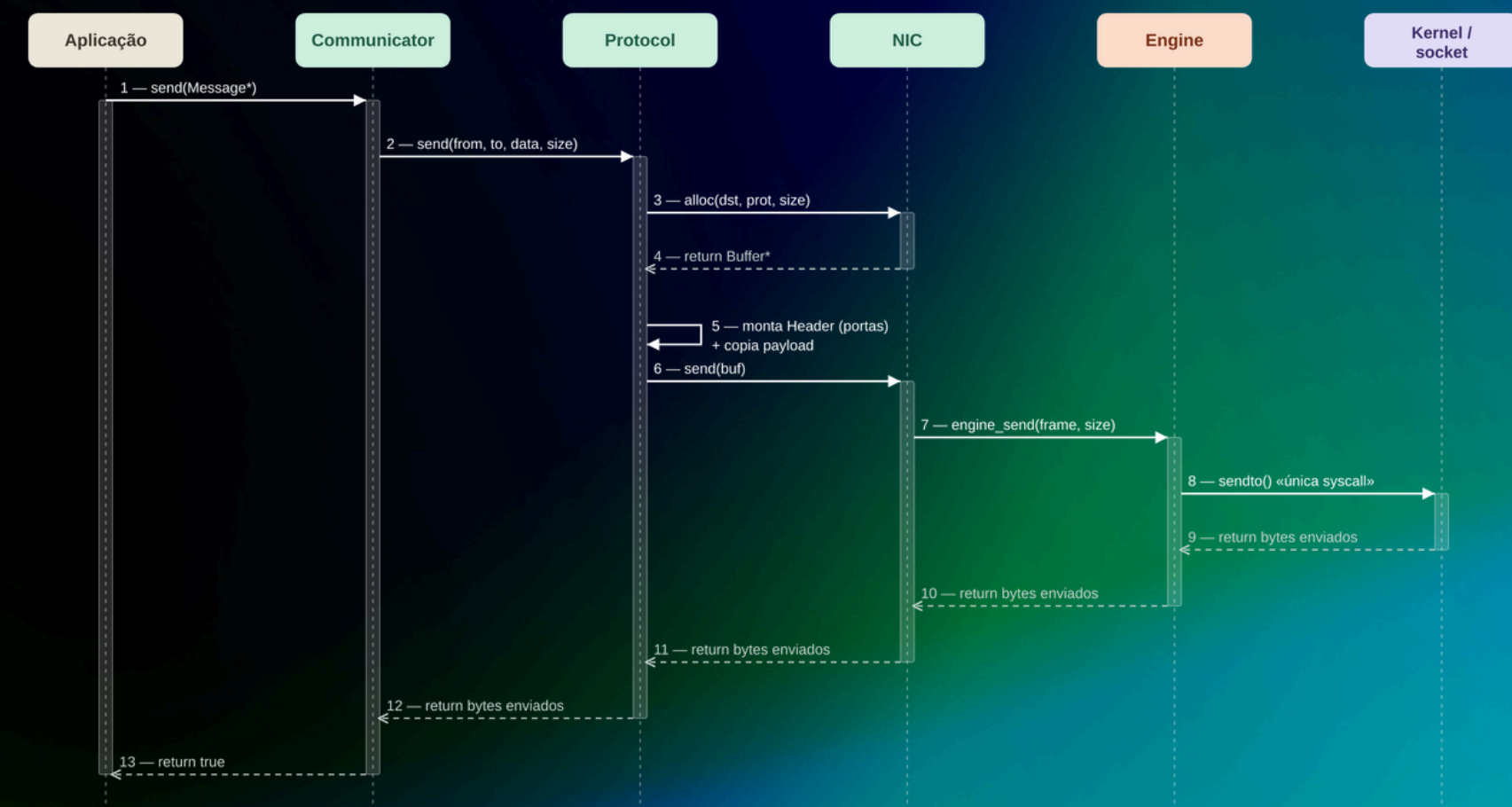
Carro 3 — receptor

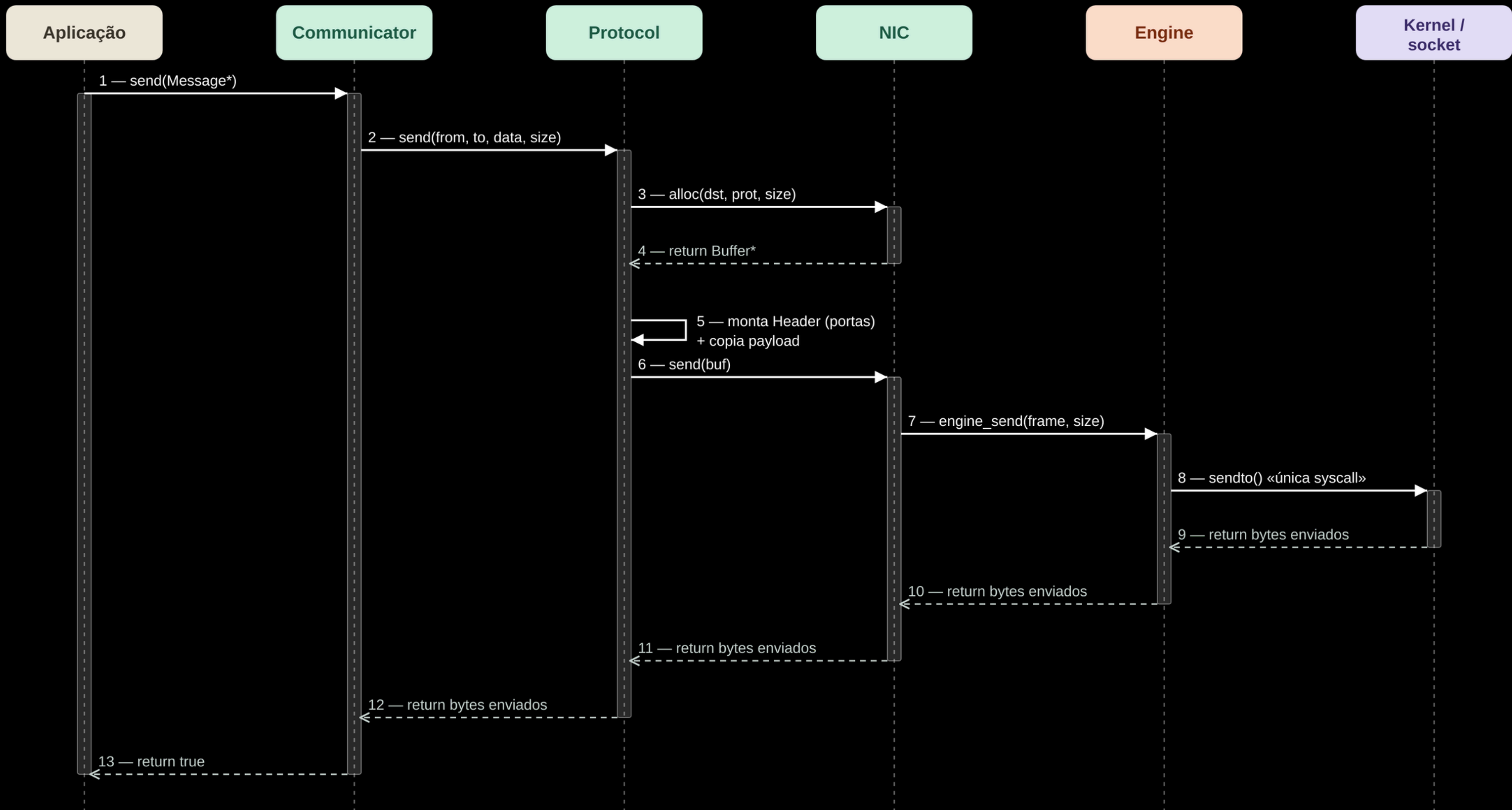
▲ recepção



SEQUÊNCIA DE ENVIO

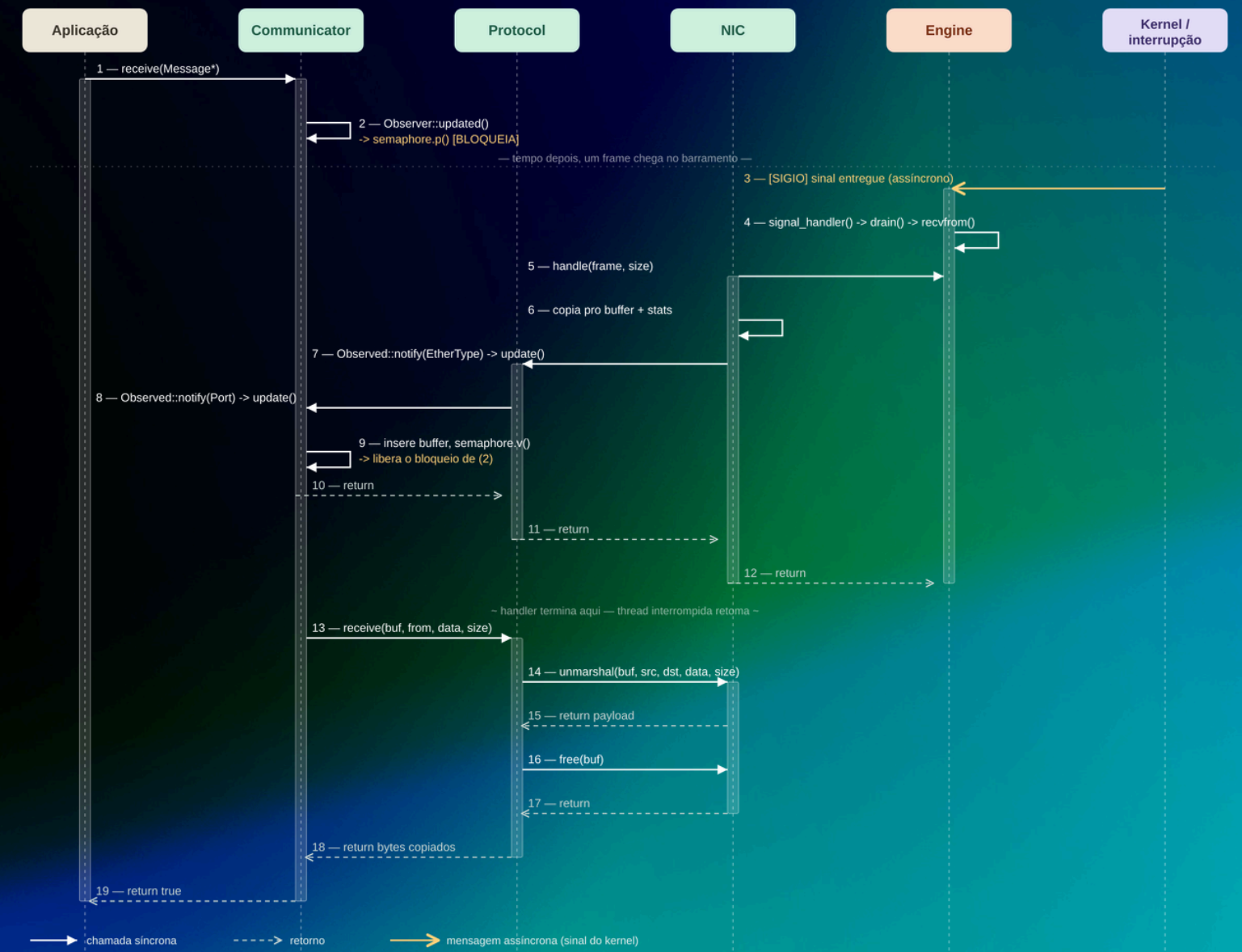
- Envio é síncrono do início ao fim — cada camada espera a de baixo retornar antes de devolver o controle
- Protocol é quem monta o cabeçalho (portas) e copia o payload — NIC só vê bytes prontos pra sair
- `engine_send()` → `sendto()` é a única syscall de todo o caminho

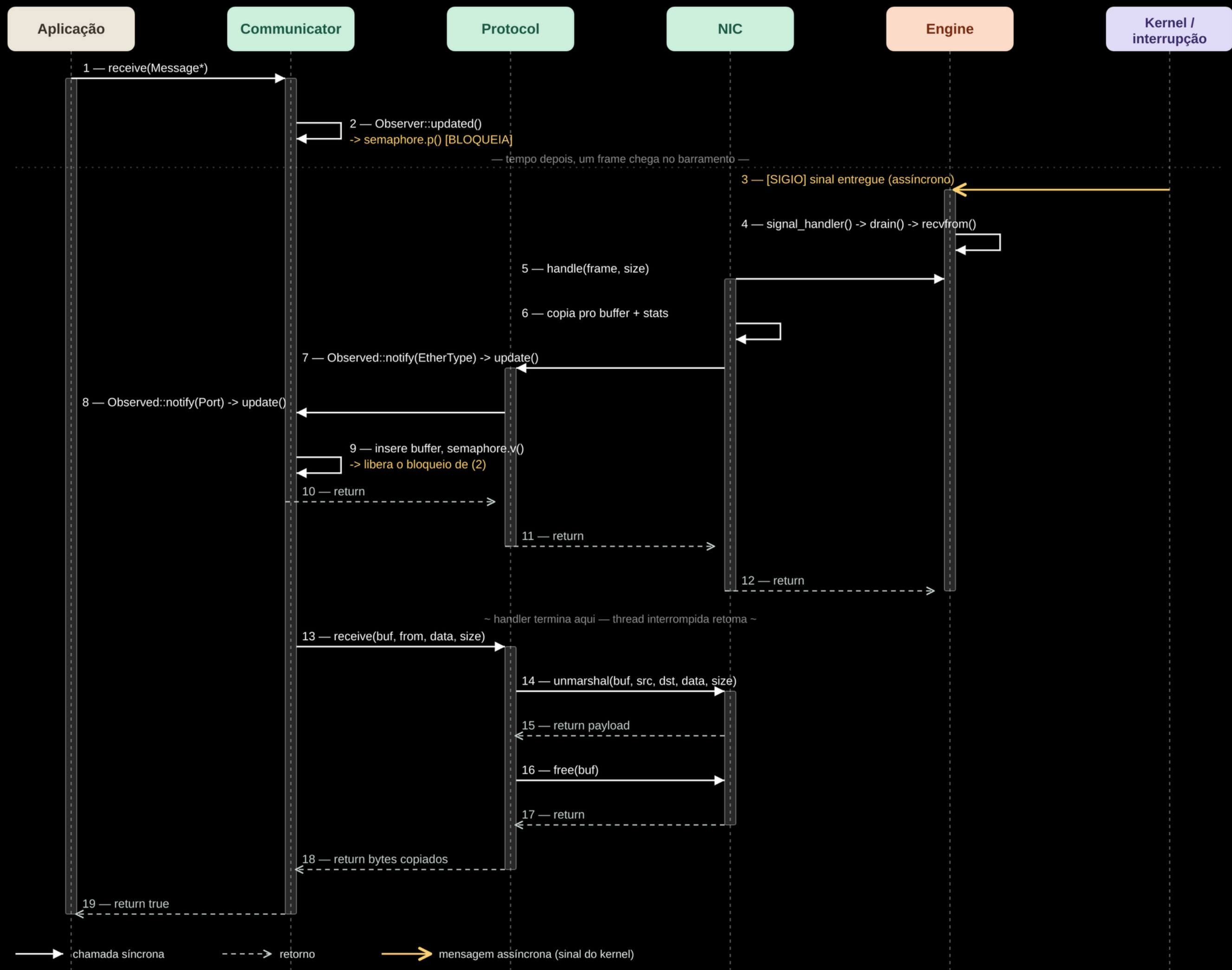




SEQUÊNCIA DE RECEPÇÃO

- Dois fluxos que se cruzam no tempo: a aplicação já está bloqueada esperando quando o sinal chega, minutos ou milissegundos depois
- O sinal (SIGIO) é mensagem assíncrona de verdade — sem retorno, sem sincronismo com quem chamou receive()
- semaphore.v() dentro do handler é o que destrava o semaphore.p() que já estava esperando — é aí que os dois fluxos se encontram
- Depois que o handler termina, o resto (unmarshal, free) roda na thread da aplicação, já fora do contexto de sinal





DECISÃO DE DESTAQUE

Recepção por Sinal POSIX

- SIGIO, não SIGRTMIN — sinal de tempo real enfileira, mas quem garante zero perda é o laço de `drain()`, não a fila do sinal
- Medido, não só argumentado: 50 frames em rajada geraram 1 sinal pendente, e `drain()` entregou os 50 numa única entrada no handler



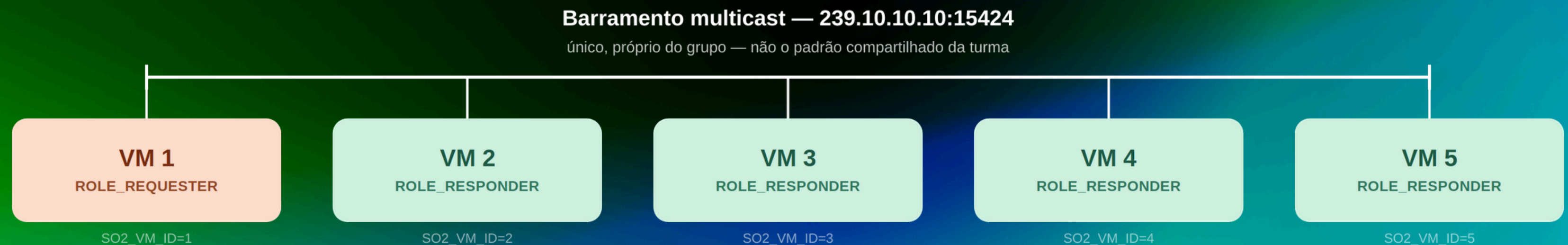
OUTRAS DECISÕES DE PROJETO

- EtherType 0x88B5 — faixa IEEE reservada pra uso experimental, não valor arbitrário. O filtro do kernel já elimina ruído: numa VM ociosa, 8 de 11 frames capturados eram IPv6 do próprio kernel
- Listas lock-free, sem alocação — mesma restrição do sinal (2.1) se aplica aqui: mutex e malloc não são signal-safe. Verificado: 58.937 sinais, 176.811 itens, zero perdidos
- Barramento multicast próprio do grupo (239.10.10.10:15424), não o padrão compartilhado da turma



TOPOLOGIA DE TESTE

- 5 VMs QEMU, todas usando um binário único — o papel (sender/receiver) vem de SO2_VM_ID, exportado pelo /init de cada VM
- VM 1 é a requester: envia REQUEST em broadcast.
- VMs 2–5 são as responders: cada uma devolve uma RESPONSE por REQUEST recebido — o par pergunta/resposta é o que permite medir latência de ida e volta



VALIDAÇÃO

TESTES AUTOMATIZADOS

- test-support: 29/29; test-stack: 23/23; test-protocol: 23/23; test-engine: 34/34
 - Total: 109 verificações no total, zero falhas
- test-engine roda em 2 níveis: sem privilégio (caminho de falha, engine inválida) e com CAP_NET_RAW (socket real na interface lo)
- Medido, não assumido: rajada de 50 frames → 1 sinal → drain() entrega os 50, zero perdidos — confirmado de novo no log desta execução



AVALIAÇÃO DE DESEMPENHO

- Latência calculada automaticamente pelo make
 - 25 REQUEST pareadas com 100 RESPONSE, 20 amostras de warm-up excluídas
 - 80 amostras válidas
- Round-trip REQUEST → RESPONSE (um único clock, no host):
 - **média** 3,681 ms · **mediana** 3,671 ms
 - **mínimo** 1,017 ms · **máximo** 9,061 ms
 - **p95** 7,996 ms · **desvio padrão** 1,901 ms
- Por veículo respondente:
 - VM3 3,014 ms · VM2 3,156 ms · VM5 3,780 ms · VM4 4,776 ms (20 amostras cada)



OBRIGADO PELA ATENÇÃO

GRUPO M10:

ANDRÉ FILIPE MARTINS

ARTUR TRIBECK FERREIRA TOMAZ

JOÃO PEDRO DE OLIVEIRA ANDERS